CERTIK

Lista Dao (Surfin) - audit

Preliminary Comments

CertiK Assessed on Sept 17th, 2026



CertiK Assessed on Sept 17th, 2026

Lista Dao (Surfin) - audit

These preliminary comments were prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
Staking	EVM Compatible	Manual Review, Static Analysis

LANGUAGE	TIMELINE
Solidity	Preliminary comments published on 08/06/2026

Vulnerability Summary



26	10	3	11	0	2
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined	Pending

1	Centralization	1 Acknowledged	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
0	Major		Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
0	Medium		Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
4	Minor	2 Resolved, 1 Acknowledged, 1 Pending	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
21	Informational	8 Resolved, 3 Partially Resolved, 9 Acknowledged, 1 Pending	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.
0	Discussion		The impact of the issue is yet to be determined, hence requires further clarifications from the project team.

TABLE OF CONTENTS | LISTA DAO (SURFIN) - AUDIT

Audit Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Review Notes

Core Contracts

CreditFundBase

FlexEarnPool

LockedEarnPool

SurfinAdapter

InterestDistributor

External Dependencies

Privileged Functions

Findings

LDS-01 : Centralization Related Risks

LDS-02 : `finishWithdraw` Can Overfund A Pool By Counting Confirmed-But-Unclaimed Withdrawals

LDS-03 : Adapter Rewiring Can Split Pool Accounting From Asset Custody

LDS-04 : Missing Minimum Term Checks For Penalty And Auto-Renew Windows

LDS-05 : Per-Address `dailyLimit` Is Bypassed By Depositing To Multiple Receivers

LDS-06 : Potential Incompatible Asset Configuration Across Surfin Contracts

LDS-07 : Locked Early-Redemption Limits Use Inconsistent Gross And Net Amounts

LDS-08 : Cancelled Withdrawals Do Not Refund The Consumed Daily-Withdraw Limit

LDS-09 : Withdrawal Requests Identified By Unstable Array Index

LDS-10 : `accruedFee` Comment Says "Manager Only" But `claimFee` Is BOT-Gated

LDS-11 : Unpaginated User State Getters Can Become Unusable

LDS-12 : Deposits Are Not Paused During Initial Adapter And Limit Setup

LDS-13 : Minimum Withdrawal Restriction Bypass Via Request Cancellation

LDS-14 : Misleading Default Waiting Period Comment

LDS-15 : One Consumed Row Can Revert Unrelated Batch Operations

LDS-16 : Incompatibility With Deflationary Tokens (Non-Standard ERC20 Token)

LDS-17 : Front-Running Withdrawal Cancellation Can Revert BOT Funding

LDS-18 : BOT Renewal Bypasses Entry-Side Controls

LDS-19 : Early Redemption Payout Is Fixed Even If Funding Arrives After Maturity

LDS-20 : Early Redemption Does Not Forfeit Merkle-Distributed Interest

LDS-21 : Cancelling Partially Funded Flex Withdrawals Can Temporarily Park Adapter Liquidity In The Flex Pool

LDS-23 : Flex Withdrawals Can Revert Stale Interest Funding

LDS-24 : Renewed Positions Can Bypass Auto-Renew Cap

LDS-25 : Auto-Renew Positions Lack Maturity Withdrawal Fallback

LDS-26 : Discussion On Upgrading To New Implementation Can Zero New Storage Variables

LDS-28 : `settleRecall` Uses A Stale Floor And Rejects Fully Funded Recalls

Appendix

Disclaimer

CODEBASE | LISTA DAO (SURFIN) - AUDIT

Repository

https://github.com/lista-dao/lista-new-contracts/

Commit

0db2041e6ebb8386c5f5ffdba735cc4d02fb78c8

50c8c4078690a7766276e16ca8be4bdc47a39d60

AUDIT SCOPE | LISTA DAO (SURFIN) - AUDIT

lista-dao/lista-new-contracts
CreditFundBase.sol
SurfinAdapter.sol
FlexEarnPool.sol
InterestDistributor.sol
LockedEarnPool.sol
src/surfin/CreditFundBase.sol
src/surfin/LockedEarnPool.sol
src/surfin/SurfinAdapter.sol
src/surfin/FlexEarnPool.sol
src/surfin/InterestDistributor.sol

APPROACH & METHODS | LISTA DAO (SURFIN) - AUDIT

This audit was conducted for Lista Dao to evaluate the security and correctness of the smart contracts associated with the Lista Dao (Surfin) - audit project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | LISTA DAO (SURFIN) - AUDIT

The **Lista Dao (Surfin)** protocol provides flexible and locked credit fund products that route user principal through shared pool accounting, an adapter-held liquidity layer, and off-pool cumulative Merkle interest distribution.

Core Contracts

CreditFundBase

The contract provides the shared accounting and administrative framework used by the Surfin flexible and locked earn pools. It stores the underlying asset, adapter address, pool metadata, withdrawal request queues, batch-level withdrawal totals, daily withdrawal limits, minimum deposit and withdrawal thresholds, and deposit pause state. Deposits are implemented by child contracts, while principal withdrawals follow a common batch process: user requests are grouped by day, the adapter later funds confirmed batches through `finishWithdraw()`, and users or authorized bots claim confirmed withdrawals from the pool. The base contract also exposes shared controls for setting minimums, daily limits, deposit pause status, adapter address, pausing, unpausing, emergency withdrawals, and UUPS upgrades. The design separates product-specific principal accounting from shared withdrawal settlement and administrative control logic.

FlexEarnPool

The contract implements the flexible Surfin earn product. User principal is represented by an internal 1:1 LP-style balance rather than a standard ERC20 token contract, with deposits increasing `balanceOf` and `totalSupply` and withdrawal requests burning the corresponding balance. Deposited assets are transferred directly from the depositor to the adapter, so the pool primarily maintains user accounting and withdrawal queue state. Withdrawals are not paid instantly by the pool; instead, requests are placed into the shared daily batch queue inherited from `CreditFundBase`, and the adapter later supplies funds to confirm batches. Users may cancel unconfirmed withdrawal requests, which restores the burned internal balance. Interest is not accrued or paid inside the pool and is handled separately by the `InterestDistributor`.

LockedEarnPool

The contract implements Surfin's fixed-term earn product using per-user positions and manager-configured cohorts. Each deposit creates a position with principal, cohort identifier, deposit time, auto-renew setting, and closed status. Cohorts define the term length, deposit deadline, maturity time, and enablement status, allowing multiple positions to share the same settlement schedule. Deposited assets are transferred directly to the adapter, while the pool tracks outstanding principal. Users may request early redemption before maturity, subject to an early-redemption payout formula, or request withdrawal after maturity when auto-renew is disabled. Bots may batch maturity withdrawals and renew matured auto-renew positions into new cohorts. Confirmed withdrawal proceeds may also be reinvested into a new locked cohort instead of being claimed. Interest remains outside the pool and is distributed separately through the Merkle distributor.

SurfinAdapter

The contract acts as the shared liquidity and operational adapter for the flexible and locked pools. Both pools forward user deposits to the adapter, which holds idle asset balances and can deploy available funds to a designated Surfin wallet. The adapter tracks deployed book value, accrued protocol fee amounts, a hard-floor liquidity reserve over both pools' live and

pending principal, and references to the flex pool, locked pool, Surfin wallet, fee receiver, and interest distributor. Managers can deploy funds, settle recalled funds, update key addresses, fund interest distribution, and configure the reserve floor rate. Bots can fund pool withdrawal batches and claim accrued fees to the configured receiver. The adapter is also the funding source for the `InterestDistributor`, using allowances and `notifyReward()` to move interest into the distributor before users claim.

InterestDistributor

The contract distributes Surfin interest through a cumulative Merkle-claim model. The contract stores a single interest token, the active Merkle root, a pending Merkle root, per-account cumulative claimed amounts, and a configurable waiting period. Interest is funded by an authorized funder, expected to be the Surfin adapter, through `notifyReward()`. Users can claim the difference between their Merkle-proven cumulative entitlement and their previously claimed amount, and claims may also be submitted in batches. Root updates follow a two-step process in which a bot sets a pending root, waits for the configured delay, and then accepts it as the active root.

External Dependencies

The **Lista Dao (Surfin)** protocol relies on a few external dependencies to fulfill the needs of its business logic.

The following are third-party dependency contracts used within the contract:

- @openzeppelin/contracts-upgradeable/access/extensions/AccessControlEnumerableUpgradeable.sol
- @openzeppelin/contracts/proxy/utils/UUPSUpgradeable.sol
- @openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol
- @openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol
- @openzeppelin/contracts/token/ERC20/IERC20.sol
- @openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol
- @openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol
- @openzeppelin/contracts/utils/cryptography/MerkleProof.sol
- @openzeppelin/contracts/utils/math/Math.sol

It is assumed that these functions are trusted and implemented properly within the whole project.

It is important to note that the audit scope excludes tokens not created by this platform; these are treated as third-party dependencies. Similarly, external contracts such as the asset token, Surfin receiving wallet, which were created by third parties or are outside the reviewed scope, are also outside the scope of this review.

Furthermore, the off-chain logic, including wallet receiving, off-chain fund deployment and recall flow, interest accounting, Merkle-tree computation, and bot/keeper logic, is considered as black boxes. It is assumed to be correctly deployed, configured, and non-malicious.

Privileged Functions

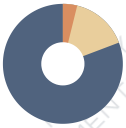
In the **Lista Dao (Surfin)** protocol, some privileged roles are adopted to ensure the dynamic runtime updates of the project, which are specified in the **Centralization Related Risks** finding.

The advantage of those privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to best serve the community.

It is also worth noting the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the private keys of the privileged accounts are compromised, it could lead to devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plan to invoke the aforementioned functions should also be considered to move to the execution queue of the `Timelock` contract.

FINDINGS | LISTA DAO (SURFIN) - AUDIT



26	0	1	0	0	4	21	0
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational	Discussion

This report has been prepared for Lista Dao to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 26 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
LDS-01	**Centralization Related Risks**	**Centralization**	**Centralization**	**Acknowledged**
LDS-02	`finishWithdraw` Can Overfund A Pool By Counting Confirmed-But-Unclaimed Withdrawals	Inconsistency	Minor	Resolved
LDS-03	Adapter Rewiring Can Split Pool Accounting From Asset Custody	Logical Issue	Minor	Pending
LDS-04	Missing Minimum Term Checks For Penalty And Auto-Renew Windows	Logical Issue	Minor	Resolved
LDS-05	Per-Address `dailyLimit` Is Bypassed By Depositing To Multiple Receivers	Design Issue	Minor	Acknowledged
LDS-06	Potential Incompatible Asset Configuration Across Surfin Contracts	Logical Issue	Informational	Resolved
LDS-07	Locked Early-Redemption Limits Use Inconsistent Gross And Net Amounts	Inconsistency, Logical Issue	Informational	Resolved
LDS-08	Cancelled Withdrawals Do Not Refund The Consumed Daily-Withdraw Limit	Design Issue	Informational	Acknowledged
LDS-09	Withdrawal Requests Identified By Unstable Array Index	Design Issue	Informational	Resolved
LDS-10	`accruedFee` Comment Says "Manager Only" But `claimFee` Is BOT-Gated	Coding Issue	Informational	Resolved

ID	Title	Category	Severity	Status
LDS-11	Unpaginated User State Getters Can Become Unusable	Coding Style	Informational	Acknowledged
LDS-12	Deposits Are Not Paused During Initial Adapter And Limit Setup	Logical Issue	Informational	Acknowledged
LDS-13	Minimum Withdrawal Restriction Bypass Via Request Cancellation	Logical Issue	Informational	Resolved
LDS-14	Misleading Default Waiting Period Comment	Inconsistency	Informational	Resolved
LDS-15	One Consumed Row Can Revert Unrelated Batch Operations	Logical Issue	Informational	Resolved
LDS-16	Incompatibility With Deflationary Tokens (Non-Standard ERC20 Token)	Volatile Code	Informational	Acknowledged
LDS-17	Front-Running Withdrawal Cancellation Can Revert BOT Funding	Logical Issue	Informational	Partially Resolved
LDS-18	BOT Renewal Bypasses Entry-Side Controls	Design Issue	Informational	Partially Resolved
LDS-19	Early Redemption Payout Is Fixed Even If Funding Arrives After Maturity	Design Issue	Informational	Acknowledged
LDS-20	Early Redemption Does Not Forfeit Merkle-Distributed Interest	Logical Issue, Design Issue	Informational	Acknowledged
LDS-21	Cancelling Partially Funded Flex Withdrawals Can Temporarily Park Adapter Liquidity In The Flex Pool	Logical Issue, Design Issue	Informational	Resolved
LDS-23	Flex Withdrawals Can Revert Stale Interest Funding	Denial of Service	Informational	Partially Resolved
LDS-24	Renewed Positions Can Bypass Auto-Renew Cap	Logical Issue	Informational	Acknowledged
LDS-25	Auto-Renew Positions Lack Maturity Withdrawal Fallback	Design Issue	Informational	Acknowledged

ID	Title	Category	Severity	Status
LDS-26	Discussion On Upgrading To New Implementation Can Zero New Storage Variables	Design Issue, Coding Issue	Informational	Acknowledged
LDS-28	`settleRecall` Uses A Stale Floor And Rejects Fully Funded Recalls	Logical Issue	Informational	Pending

LDS-01 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	Centralization		Acknowledged

Description

In the contract `FlexEarnPool`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `grantRole()`
- `revokeRole()`
- `setAdapter()`
- `emergencyWithdraw()`
- `upgradeToAndCall()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to grant or revoke privileged roles, redirect the protocol adapter, rescue tokens held by the pool, or upgrade the contract implementation.

In the contract `FlexEarnPool`, the role `BOT` has authority over the following functions:

- `claimWithdraw()`

Any compromise to the `BOT` account may allow an attacker to trigger confirmed withdrawal claims on behalf of users and mutate withdrawal request state.

In the contract `FlexEarnPool`, the role `MANAGER` has authority over the following functions:

- `setMinDeposit()`
- `setMinWithdraw()`
- `setDailyLimit()`
- `setDepositPaused()`
- `unpause()`

Any compromise to the `MANAGER` account may allow an attacker to change deposit and withdrawal thresholds, alter daily withdrawal limits, block deposits, or resume protocol activity after a pause.

In the contract `FlexEarnPool`, the role `PAUSER` has authority over the following functions:

- `pause()`

Any compromise to the `PAUSER` account may allow an attacker to pause protocol operations that depend on the global pause state, including deposits, withdrawal requests, cancellations, and claims.

In the contract `LockedEarnPool`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `grantRole()`
- `revokeRole()`
- `setAdapter()`
- `emergencyWithdraw()`
- `upgradeToAndCall()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to grant or revoke privileged roles, redirect the protocol adapter, rescue tokens held by the pool, or upgrade the contract implementation.

In the contract `LockedEarnPool`, the role `BOT` has authority over the following functions:

- `claimWithdraw()`
- `batchRequestMaturityWithdraw()`
- `renewPosition()`
- `setCohort()`

Any compromise to the `BOT` account may allow an attacker to trigger confirmed withdrawal claims on behalf of users, queue matured withdrawals on behalf of users, renew matured auto-renew positions into selected cohorts, or configure cohort terms, deposit deadlines, maturity dates, and enabled status, including for cohorts already referenced by existing positions.

In the contract `LockedEarnPool`, the role `MANAGER` has authority over the following functions:

- `setMinDeposit()`
- `setMinWithdraw()`
- `setDailyLimit()`
- `setDepositPaused()`
- `unpause()`
- `setPenaltyRate()`

Any compromise to the `MANAGER` account may allow an attacker to change deposit and withdrawal thresholds, alter daily withdrawal limits, block deposits and locked early redemptions, resume protocol activity after a pause, or change the early-redemption penalty rate applied within the penalty window.

In the contract `LockedEarnPool`, the role `PAUSER` has authority over the following functions:

- `pause()`

Any compromise to the `PAUSER` account may allow an attacker to pause protocol operations that depend on the global pause state, including deposits, early redemptions, maturity withdrawals, toggles, reinvests, and claims.

In the contract `SurfinAdapter`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `grantRole()`
- `revokeRole()`
- `emergencyWithdraw()`

- `upgradeToAndCall()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to grant or revoke privileged roles, rescue tokens held by the adapter, or upgrade the adapter implementation.

In the contract `SurfinAdapter`, the role `MANAGER` has authority over the following functions:

- `deployToSurfin()`
- `settleRecall()`
- `fundInterest()`
- `unpause()`
- `setFloorRate()`
- `setSurfinWallet()`
- `setInterestDistributor()`
- `setFeeReceiver()`

Any compromise to the `MANAGER` account may allow an attacker to deploy funds to the Surfin wallet, reset deployed book value and fee accounting, fund the interest distributor, resume operations, change reserve requirements, redirect custody and distribution addresses, or change the fee receiver.

In the contract `SurfinAdapter`, the role `BOT` has authority over the following functions:

- `finishFlexWithdraw()`
- `finishLockedWithdraw()`
- `claimFee()`

Any compromise to the `BOT` account may allow an attacker to control when pool withdrawal queues are repaid from idle funds or transfer accrued fees to the configured fee receiver.

In the contract `SurfinAdapter`, the role `PAUSER` has authority over the following functions:

- `pause()`

Any compromise to the `PAUSER` account may allow an attacker to pause outbound adapter operations such as deployment to Surfin and recall settlement.

In the contract `InterestDistributor`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `grantRole()`
- `revokeRole()`
- `upgradeToAndCall()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to grant or revoke privileged roles or upgrade the distributor implementation.

In the contract `InterestDistributor`, the role `FUNDER` has authority over the following functions:

- `notifyReward()`

Any compromise to the `FUNDER` account may allow an attacker to control reward funding into the distributor, affecting the availability and timing of user interest payouts.

In the contract `InterestDistributor`, the role `BOT` has authority over the following functions:

- `setPendingMerkleRoot()`
- `acceptMerkleRoot()`

Any compromise to the `BOT` account may allow an attacker to stage and accept Merkle roots that determine user interest claim entitlements.

In the contract `InterestDistributor`, the role `MANAGER` has authority over the following functions:

- `revokePendingMerkleRoot()`
- `changeWaitingPeriod()`
- `emergencyWithdraw()`
- `unpause()`

Any compromise to the `MANAGER` account may allow an attacker to revoke pending reward roots, change the root acceptance delay, withdraw all tokens from the distributor in an emergency path, or resume claims and root-management operations.

In the contract `InterestDistributor`, the role `PAUSER` has authority over the following functions:

- `pause()`

Any compromise to the `PAUSER` account may allow an attacker to pause reward funding, claims, and root-management operations.

As with standard OpenZeppelin `AccessControl`, any privileged account may also call `renounceRole()` for roles it holds. It only allows the caller to relinquish its own privileges, but a compromised EOA could use it to disrupt operations by renouncing critical roles.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key

management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[Lista Dao, 08/18/2026]: Issue acknowledged. I won't make any changes for the current version.

Mainnet custody: DEFAULT_ADMIN_ROLE (upgrades, setAdapter, emergencyWithdraw, adminTopUp) to a TimeLock; MANAGER to a multisig Safe; PAUSER and MANAGER never collapsed.

[CertiK, 08/26/2026]: In the new version b306c579c00b9c2a24f39aad61b9ca1807d19a8d, `adminTopUp()` was added to `CreditFundBase` and is controlled by `DEFAULT_ADMIN_ROLE`, affecting both `FlexEarnPool` and `LockedEarnPool`. `LockedEarnPool.setCohort()` is no longer exclusively controlled by `BOT`; `MANAGER` can now also call it in certain update scenarios.

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to top up pool withdrawal funding through `adminTopUp()`. Any compromise to the `MANAGER` account may allow an attacker to modify existing cohort parameters through `setCohort()` in the permitted update scenarios.

It is suggested to implement the aforementioned methods to avoid centralized failure. Also, CertiK strongly encourages the

project team to periodically revisit the private key security management of all addresses related to centralized roles.

LDS-02 `finishWithdraw` Can Overfund A Pool By Counting Confirmed-But-Unclaimed Withdrawals

Category	Severity	Location	Status
Inconsistency	Minor	CreditFundBase.sol (07/27-50c8c4): 167	Resolved

Description

`CreditFundBase.finishWithdraw` caps `withdrawQuota` against `totalPendingWithdraw`. However, `totalPendingWithdraw` includes both unconfirmed withdrawal requests and confirmed-but-unclaimed requests.

Once a batch is confirmed, the pool already holds the cash needed to pay those users, but the confirmed amount remains in `totalPendingWithdraw` until users claim. As a result, the cap can be loose by the confirmed-but-unclaimed amount.

For example, if a 100-unit batch has already been confirmed but not claimed, `totalPendingWithdraw` is still 100 while `withdrawQuota` is 0. A further BOT call pushing another 100 units will pass:

```
withdrawQuota += amount;
require(withdrawQuota <= totalPendingWithdraw, "quota exceeds pending");
```

No new batch is confirmed, and the pool now holds 200 units for only 100 units of confirmed user claims. The surplus remains as `withdrawQuota` and can only be consumed by future batch confirmations or recovered through privileged emergency flows.

The impact is limited because funding is BOT/adapter-controlled and adapter-side checks still protect `accruedFee` and `hardFloor`. Still, the current cap does not fully enforce the stated intent of preventing over-pushed funds from being relocated from the adapter into a pool.

Recommendation

Cap new funding against the pool's remaining unfunded withdrawal need, excluding confirmed-but-unclaimed claims that are already backed by pool cash. Ensure existing pool cash is included in the cap so surplus funds cannot be pushed into the pool beyond its real outstanding funding requirement.

Alleviation

[Lista Dao, 08/19/2026]:

The team heeded the advice and resolved the issue in commit 8b0897f63e432993a08abe26338ca3712206e2dc.

LDS-03 Adapter Rewiring Can Split Pool Accounting From Asset Custody

Category	Severity	Location	Status
Logical Issue	Minor	CreditFundBase.sol (07/27-50c8c4): 234~239	Pending

Description

Pool deposits record user principal in the pool while transferring the underlying asset directly to the current `adapter`. However, `setAdapter()` can replace the adapter address with only nonzero and different-address checks. It does not require the pool to have no live principal, no pending withdrawals, or no residual withdrawal quota, and it does not perform any custody migration from the old adapter to the new one.

After the adapter is changed, `finishWithdraw()` accepts funding only from the new adapter. As a result, principal previously sent to the old adapter or to a temporary placeholder adapter is no longer in the normal withdrawal funding path unless operators separately migrate or refund those funds. Existing users may be left waiting for the new adapter to fund obligations backed by assets it never received, or the new adapter's liquidity may be used to satisfy claims whose backing assets remain elsewhere.

Impact

User withdrawals can be delayed or blocked until operators manually reconcile custody. Alternatively, liquidity held by the new adapter may be used to pay claims backed by assets still held by the old adapter or placeholder address, causing cross-adapter liability contamination.

Recommendation

Prevent adapter changes while the pool has unresolved accounting. At minimum, require `totalPrincipal() == 0`, `totalPendingWithdraw == 0`, and `withdrawQuota == 0` before allowing `setAdapter()` to update the adapter.

If live adapter migration is required, implement a dedicated migration flow that first reconciles or transfers custody from the old adapter to the new adapter, then updates the pool adapter only after the required backing assets and outstanding withdrawal state are accounted for.

During deployment, keep deposits disabled until the final adapter is wired and verified.

Alleviation

[Lista Dao, 08/18/2026]:

Fix commit: (f93271e, 2a1a67b).

[CertiK, 08/24/2026]:

The fix requires `totalPrincipal() == 0` before `setAdapter()` can run, so any Flex LP minted to an address that cannot or will not call `requestWithdraw()` can keep `totalSupply` nonzero forever and permanently block adapter replacement.

LDS-04 Missing Minimum Term Checks For Penalty And Auto-Renew Windows

Category	Severity	Location	Status
Logical Issue	Minor	LockedEarnPool.sol (07/27-50c8c4): 11, 231, 385	Resolved

Description

`LockedEarnPool` is described as a longer-tenor locked product, such as 3M+ or 6M+ terms. However, its early redemption and auto-renewal controls are governed by two fixed windows that do not depend on the actual cohort duration:

- `PENALTY_WINDOW` is fixed at 30 days from `depositTime`, during which early redemption applies a flat principal haircut.
- `AUTO_RENEW_LOCK_WINDOW` is fixed at 32 days before `maturityTime`, during which `toggleAutoRenew` is disabled.

The issue is that `setCohort` only requires `termDays > 0` and checks that `maturityTime` falls between the nominal term end and that end plus `MAX_ALIGN_WINDOW` of 31 days. It does not enforce that the resulting lock period is longer than either of the fixed control windows.

```
uint256 public constant PENALTY_WINDOW = 30 days;
uint256 public constant AUTO_RENEW_LOCK_WINDOW = 32 days;

require(termDays > 0, "term is zero");
uint256 nominalEnd = depositDeadline + termDays * 1 days;
require(maturityTime >= nominalEnd, "maturity before term end");
require(maturityTime <= nominalEnd + MAX_ALIGN_WINDOW, "maturity too late");

if (block.timestamp < depositTime + PENALTY_WINDOW) { /* penalize */ }

require(
  block.timestamp + AUTO_RENEW_LOCK_WINDOW < cohorts[pos.cohortId].maturityTime,
  "auto renew locked (T-30)"
);
```

For users who deposit close to `depositDeadline`, the remaining time to maturity is approximately `termDays + (maturityTime - nominalEnd)`. If this period is shorter than the fixed windows, every valid early redemption before maturity may still fall within `PENALTY_WINDOW`, meaning the principal penalty always applies. Separately, because `toggleAutoRenew` requires `now + 32 days < maturityTime`, the auto-renewal choice made at deposit may remain immutable for the entire position. Although `renewPosition` can still execute at maturity if auto-renewal was already enabled, users may be unable to turn it off, or enable it, after depositing.

Recommendation

Consider requiring `maturityTime > depositDeadline + AUTO_RENEW_LOCK_WINDOW` in `setCohort()`, to ensure users have a valid window to update their auto-renewal preference.

Alleviation

[CertiK, 08/24/2026]:

The team heeded the advice and resolved the issue by requiring `termDays > AUTO_RENEW_LOCK_WINDOW / 1 days` in `setCohort()`, so `maturityTime` is always more than 32 days after `depositDeadline`. Last-moment depositors still have a window to toggle auto-renew and to redeem outside the 30-day penalty.

Changes have been reflected in commit 6d12c34

LDS-05 Per-Address `dailyLimit` Is Bypassed By Depositing To Multiple Receivers

Category	Severity	Location	Status
Design Issue	Minor	CreditFundBase.sol (07/27-50c8c4): 330	Acknowledged

Description

`_consumeDailyLimit` keys the throttle on `msg.sender` (`dailySubmitted[day][user]`), but both deposit entrypoints credit an arbitrary beneficiary — `FlexEarnPool.deposit(amount, receiver)` and `LockedEarnPool.deposit(cohortId, amount, receiver, autoRenew)` . Splitting capital across N controlled addresses at deposit time gives N × `dailyLimit` per day. `minDeposit` does not help, since each sub-account is sized at `dailyLimit` , not at dust.

Recommendation

Double confirm if this design is intended.

Alleviation

[Lista Dao, 08/18/2026]:

Issue acknowledged. I won't make any changes for the current version.

LDS-06 Potential Incompatible Asset Configuration Across Surfin Contracts

Category	Severity	Location	Status
Logical Issue	Informational	CreditFundBase.sol (07/27-50c8c4): 234; InterestDistributor.sol (07/27-50c8c4): 92~93; SurfinAdapter.sol (07/27-50c8c4): 101~102, 308	Resolved

Description

The Surfin system assumes that the pools, adapter, and interest distributor all operate on the same underlying asset. However, several configuration paths only validate address presence and do not enforce asset consistency across connected contracts.

SurfinAdapter.constructor():

```
  /* CONSTRUCTOR */
  /// @custom:oz-upgrades-unsafe-allow constructor
  /// @param _asset The address of the asset token (USDT).
  constructor(address _asset) {
    require(_asset != address(0), "asset is zero address");
    _disableInitializers();
    asset = _asset;
  }
```

SurfinAdapter.initialize():

```
require(_flexPool != address(0), "flexPool is zero address");
require(_lockedPool != address(0), "lockedPool is zero address");

flexPool = _flexPool;
lockedPool = _lockedPool;
```

SurfinAdapter.setInterestDistributor():

```
function setInterestDistributor(address _interestDistributor) external
onlyRole(MANAGER) {
  require(_interestDistributor != address(0), "interestDistributor is zero
address");
  interestDistributor = _interestDistributor;
  emit SetInterestDistributor(_interestDistributor);
}
```

CreditFundBase.setAdapter():

```
function setAdapter(address _adapter) external onlyRole(DEFAULT_ADMIN_ROLE) {
  require(_adapter != address(0), "adapter is zero address");
  require(_adapter != adapter, "same adapter");
  adapter = _adapter;
  emit SetAdapter(_adapter);
}
```

InterestDistributor.initialize():

```
require(_funder != address(0), "Invalid funder address");
require(_token != address(0), "Invalid token address");

token = _token;
_grantRole(FUNDER, _funder);
```

As a result, the system can be configured with pools whose `asset` differs from the adapter's immutable `asset`, or with an interest distributor whose `token` differs from the adapter asset used for funding. In the pool case, users may deposit one token while the adapter calculates liquidity, funds withdrawals, pays fees, and deploys capital using another token. In the distributor case, `fundInterest()` approves the adapter asset, while `notifyReward()` attempts to pull the distributor's independently configured token, which can cause interest funding to revert or remain misconfigured until the configuration is corrected.

These issues are practically reachable during deployment, migration, or privileged reconfiguration if an incorrect pool, adapter, funder, or distributor address is supplied. Once user deposits or claimable interest are involved, correcting the configuration may require privileged rescue actions, an upgrade, or a migration to restore consistent accounting.

Recommendation

Consider adding asset-consistency validation at each configuration boundary to ensure that the pools, adapter, funder, and interest distributor are wired only when they share the expected underlying asset. This validation should be enforced on-chain during initialization and later reconfiguration.

Alleviation

[Lista Dao, 08/24/2026]:

The team heeded the advice and resolved the issue in commit 2a1a67b.

LDS-07 | Locked Early-Redemption Limits Use Inconsistent Gross And Net Amounts

Category	Severity	Location	Status
Inconsistency, Logical Issue	Informational	CreditFundBase.sol (07/27-50c8c4): 327, 342; LockedEarnPool.sol (07/27-50c8c4): 149, 154, 162	Resolved

Description

Locked early redemptions apply withdrawal controls to different amount bases when an early-redemption penalty is active. The minimum-withdraw check uses the gross redeemed principal, but the daily-limit counter and queued withdrawal use the lower penalty-adjusted payout.

In `LockedEarnPool.requestEarlyRedeem`, `_checkMinWithdraw(amount, pos.principal)` runs before `_earlyRedeemPayout` reduces the payout. The function then calls `_consumeDailyLimit(msg.sender, payout)` and enqueues only `payout`. Because the shared helpers accept whichever amount basis the caller passes, they do not preserve a single withdrawal-limit invariant for this path.

```
   function requestEarlyRedeem(uint256 posId, uint256 amount) external whenNotPaused
whenDepositNotPaused nonReentrant {
    Position storage pos = userPositions[msg.sender][posId];
    require(pos.principal > 0 && !pos.closed, "invalid position");
    require(amount > 0 && amount <= pos.principal, "invalid amount");

    // min-withdraw floor with dust exit: a sub-min redeem must clear the position
@>  _checkMinWithdraw(amount, pos.principal);

    uint256 cohortId = pos.cohortId;
    require(block.timestamp < cohorts[cohortId].maturityTime, "already matured");

@>  uint256 payout = _earlyRedeemPayout(pos.depositTime, amount);

    pos.principal -= amount;
    if (pos.principal == 0) {
      pos.closed = true;
    }
    totalPrincipalAmount -= amount;

@>  _consumeDailyLimit(msg.sender, payout);
    uint256 batchId = _enqueueWithdraw(msg.sender, payout, cohortId);

    emit RequestEarlyRedeem(msg.sender, posId, amount, batchId, payout);
  }
```

A user can redeem a gross amount that satisfies `minWithdraw` while enqueueing a payout below the configured minimum. The same mismatch lets the user redeem more gross principal than the configured daily cap by absorbing the penalty, because only the reduced payout is counted against `dailySubmitted`. This is reachable during the penalty window when `penaltyRate > 0` and the locked pool has nonzero `minWithdraw` or `dailyLimit`; with the launch `0.8%` penalty and `200,000 USDT` locked daily limit, about `201,612.90 USDT` of gross principal can be redeemed while consuming only `200,000 USDT` of quota.

Recommendation

Use one explicit amount basis for each withdrawal invariant. If the limits are intended to constrain queued cash outflow, enforce the minimum-withdraw floor on `payout`; if they are intended to constrain redeemed principal, consume the daily limit with the gross `amount`. Add boundary tests for penalized early redemptions around `minWithdraw` and `dailyLimit`.

Alleviation

[CertiK, 08/24/2026]:

The team heeded the advice and resolved the issue in commit 2a1a67b.

LDS-08 | Cancelled Withdrawals Do Not Refund The Consumed Daily-Withdraw Limit

Category	Severity	Location	Status
Design Issue	Informational	CreditFundBase.sol (07/27-50c8c4): 288, 327	Acknowledged

Description

`_consumeDailyLimit` adds the requested amount to `dailySubmitted[day][user]` when a withdrawal is queued in `requestWithdraw`, but the cancel path (`cancelWithdraw` -> `_removeWithdrawRequest`) reverses the batch and pending-withdraw accounting without decrementing `dailySubmitted`. When the manager has enabled the throttle (`dailyLimit > 0`), a user who requests withdrawals up to the daily limit and then cancels them has no pending request yet still cannot submit any new withdrawal for the rest of that epoch day.

The impact is self-inflicted and recoverable. `dailySubmitted` is keyed by `msg.sender` and is only bumped by the caller's own `requestWithdraw` / `requestEarlyRedeem`; the BOT matured-withdraw path never touches it, so no one can consume another user's allowance. No funds are locked: confirmed withdrawals stay claimable and the counter resets at the next epoch day. Not refunding on cancel may also be an intentional anti-churn choice, so this is raised to confirm the intended behavior.

Recommendation

Decide the intended semantics of the throttle. If it should count only active submissions, decrement `dailySubmitted[day][user]` inside `_removeWithdrawRequest`, but only when the cancelled request was submitted in the current epoch day (`req.withdrawTime / 1 days == block.timestamp / 1 days`), so cancelling a request from a previous day cannot credit today's counter with free allowance. Otherwise, document that cancellations intentionally do not refund the daily limit.

Alleviation

[Lista Dao, 08/14/2026]:

Issue acknowledged. This is intended behavior.

LDS-09 | Withdrawal Requests Identified By Unstable Array Index

Category	Severity	Location	Status
Design Issue	Informational	CreditFundBase.sol (07/27-50c8c4): 190, 317	Resolved

Description

`claimWithdraw`, `cancelWithdraw`, and `reinvest` identify a withdrawal request only by its position `idx` in `userWithdrawalRequests[user]`. The shared helpers `_consumeConfirmedWithdraw` and `_removeWithdrawRequest` delete it with swap-and-pop: the last element is moved into the freed slot and the array shrinks. An array index is not a stable identifier, so if the array is mutated between the moment a caller reads `idx` and the moment the transaction runs, `idx` now points at a different request or is out of range.

Restricting `claimWithdraw` to the owner or the `BOT` removed the arbitrary third-party reorder vector. Two writers of a user's array remain: the trusted `BOT`, which may claim a user's confirmed request on their behalf, and the user's own in-flight index calls. If a removal lands on the exact slot another call targets (or that call targets the last slot), the second call runs against a different request of the same user, or reverts with "invalid index" / "not able to claim yet".

No funds are at risk. Every payout goes to the request owner and every request in a user's array belongs to that user, so nothing can be stolen, over-claimed, double-claimed, or stranded, and `totalPendingWithdraw` stays consistent (each request added once, removed once). The worst case is that a user's `reinvest` / `cancelWithdraw` acts on a different one of their own requests (for example locking a different confirmed payout into a term), or a transaction reverts and must be retried; the `idx` emitted in `ClaimWithdrawal` / `CancelWithdrawal` is likewise unstable for off-chain indexers. The trigger is a same-block collision, so likelihood is low.

Recommendation

Validate a caller-supplied expectation before mutating: accept the expected `batchId` (and optionally `amount`) alongside `idx` in the index-based entrypoints and `require` it matches `reqs[idx]`, turning a silent wrong-request action into a clean revert. For a full fix, assign each request a monotonic id and look it up by id instead of by array position, removing index instability entirely.

Alleviation

[CertiK, 08/24/2026]:

The team added `expectedAmount` to `claimWithdraw()`, `cancelWithdraw()`, and `reinvest()`. If swap-and-pop moves a different request into that slot, a mismatched amount now reverts instead of acting on the wrong row.

Requests are still identified by array `idx`, not a stable ID. Two of the user's own requests with the same amount can still be swapped, so the call may silently hit the other one. Other collisions revert and can be retried.

Changes have been reflected in commit ee9fc63576d5d50c42e54ccfb6f2c47c990767b4.

LDS-10 `accruedFee` Comment Says "Manager Only" But `claimFee` Is BOT-Gated

Category	Severity	Location	Status
Coding Issue	Informational	SurfinAdapter.sol (07/27-50c8c4): 42, 220	Resolved

Description

The comment on `accruedFee` (line 42) says the fee earmark is "withdrawable by manager only," but `claimFee` (line 220), the only function that pays it out, is gated to `onlyRole(BOT)`, not MANAGER. The function docstring (lines 217-219) confirms BOT is the intended caller, so the line-42 comment is the stale one.

```
// accrued Lista profit fee earmark; withdrawable by manager only
uint256 public accruedFee;
```

Impact is documentation-only. `claimFee` sends the fee to the manager-set `feeReceiver`, so a BOT caller cannot redirect funds to an arbitrary address; it can only move the accrued fee to the intended receiver. The risk is that an operator trusting the comment underestimates that the hotter, automated BOT key can trigger fee payouts, which may weaken BOT key management.

Recommendation

Update the line-42 comment to match the code: `accruedFee` is claimed by the BOT to the manager-set `feeReceiver`. If manager-only withdrawal was intended instead, change `claimFee` to `onlyRole(MANAGER)`.

Alleviation

[CertiK, 08/24/2026]:

The team heeded the advice and resolved the issue in commit a8f1ea0.

LDS-11 | Unpaginated User State Getters Can Become Unusable

Category	Severity	Location	Status
Coding Style	Informational	CreditFundBase.sol (07/27-50c8c4): 199, 278; LockedEarnPool.sol (07/27-50c8c4): 118, 318	Acknowledged

Description

The protocol exposes full-array getters for user state without pagination or bounded range access. `CreditFundBase.getUserWithdrawalRequests` copies and returns the entire `userWithdrawalRequests[user]` array, which can grow through flex withdrawal requests, locked early redemptions, and locked maturity withdrawals. `LockedEarnPool.getUserPositions` similarly copies and returns the entire `userPositions[user]` array. Large arrays can exceed practical `eth_call` gas limits or RPC response-size limits, degrading the official enumeration path for affected accounts. The position getter has an additional griefing vector because `LockedEarnPool.deposit` lets any caller choose a nonzero receiver, so an attacker can fund repeated minimum deposits to append unwanted positions to a victim's list. This does not steal funds or necessarily block index-based state-changing flows when indices are known, but it can make frontends, bots, and integrators unable to reliably discover a user's withdrawals or positions.

Recommendation

Add bounded enumeration APIs for both withdrawal requests and positions, such as count getters plus offset-and-limit range getters. Migrate UI, bot, and integration consumers to the paginated paths, while treating full-array getters only as convenience helpers for small arrays. The invariant should be that all user withdrawal requests and locked positions remain discoverable with bounded gas and response size regardless of total array length.

Alleviation

[Lista Dao, 08/14/2026]:

Issue acknowledged. I won't make any changes for the current version.

LDS-12 | Deposits Are Not Paused During Initial Adapter And Limit Setup

Category	Severity	Location	Status
Logical Issue	Informational	CreditFundBase.sol (07/27-50c8c4): 84, 107	Acknowledged

Description

Pool initialization leaves `depositPaused` false, so user entrypoints can become live as soon as a proxy is initialized, before the final adapter and launch parameters are configured.

For `FlexEarnPool`, `deposit` is externally callable immediately when the pool is unpaused and `depositPaused` is false. It mints principal first and then transfers the asset to the current `adapter`. In the provided deployment flow, the temporary adapter is the deployer address until a later `setAdapter` transaction, so a deposit included in that setup window would create pool principal while sending the cash to the placeholder address rather than the final adapter.

The same window can also allow flex deposits and withdrawal requests to use the zero defaults for `minDeposit`, `minWithdraw`, and `dailyLimit` before the manager setter transactions run. `LockedEarnPool` is narrower than originally described: locked deposits are additionally blocked unless a BOT has enabled a cohort with an open deposit deadline, so the placeholder-adapter deposit path is only reachable there if a cohort is enabled before final adapter and launch-limit setup completes.

The practical impact is a short deployment/setup race. Early flex principal can be under-backed at the final adapter until the placeholder holder forwards the received funds, and intended launch limits can be bypassed for requests already accepted during the window.

Recommendation

Initialize pools in a fail-closed state by setting `depositPaused = true` in `__CreditFundBase_init`. Only unpause deposits after the final adapter is set, launch limits are configured, and any locked cohorts intended for launch are ready. Deployment scripts should also assert the paused state before setup and only unpause as the final launch step.

Alleviation

[Lista Dao, 08/14/2026]:

Issue acknowledged. I won't make any changes for the current version.

LDS-13 Minimum Withdrawal Restriction Bypass Via Request Cancellation

Category	Severity	Location	Status
Logical Issue	Informational	FlexEarnPool.sol (07/27-50c8c4): 73~81	Resolved

Description

`requestWithdraw` enforces `minWithdraw` but permits a withdrawal below this threshold when `amount == balanceOf[msg.sender]`, intended to allow users to fully exit dust balances. Pending withdrawal requests reduce `balanceOf`, while `cancelWithdraw` restores the cancelled request amount.

A user can first request a valid withdrawal that reduces their balance to a dust amount, then request withdrawal of that remaining dust amount under the full-balance exception. They can subsequently call `cancelWithdraw` on the initial larger request, restoring its amount while leaving the sub-`minWithdraw` request pending. Thus, the exception is evaluated against a temporary, manipulable balance rather than the user's effective active position.

This bypasses the `minWithdraw` restriction and permits arbitrarily small pending withdrawal requests, potentially enabling withdrawal-queue spam or disrupting off-chain withdrawal processing.

```
function requestWithdraw(uint256 amount) external whenNotPaused nonReentrant {
    require(amount > 0, "amount is zero");
    require(balanceOf[msg.sender] >= amount, "insufficient balance");

    // min-withdraw floor with dust exit: a sub-min request must drain the balance
    _checkMinWithdraw(amount, balanceOf[msg.sender]);

    // enforce per-address daily submit limit
    _consumeDailyLimit(msg.sender, amount);

    // burn LP and enqueue
    _burn(msg.sender, amount);
    uint256 batchId = _enqueueWithdraw(msg.sender, amount, 0);

    emit RequestWithdraw(msg.sender, msg.sender, batchId, amount);
  }
```

Scenario

A user requests a withdrawal that leaves a residual balance below `minWithdraw`, then requests withdrawal of that entire residual balance using the full-balance dust exception. If the user cancels the first, larger withdrawal request afterward, its amount is restored to their balance while the sub-`minWithdraw` withdrawal remains pending, so the account no longer represents a full exit despite having an accepted dust-sized request.

Recommendation

Double check if this design is intended. If not, the team may need to track each user's effective position across both available balance and pending withdrawal requests when evaluating the full-balance exception.

Alleviation

[CertiK, 08/24/2026]:

The team heeded the advice and resolved the issue by requiring a dust-exit to have no cancellable pending request (`_hasCancellableRequest()`). A leftover sub `minWithdraw` can no longer be created by requesting a large withdrawal, dusting off the remainder, then cancelling the large one.

Changes have been reflected in commit bffd316.

LDS-14 | Misleading Default Waiting Period Comment

Category	Severity	Location	Status
Inconsistency	Informational	InterestDistributor.sol (07/27-50c8c4): 52, 99, 203	Resolved

Description

The `InterestDistributor` NatSpec states that the Merkle-root waiting period defaults to one day, but `initialize` sets `waitingPeriod` to 6 hours. Because `acceptMerkleRoot` enforces this value, a pending root can become live after only 6 hours, which does not match the documented default protection window.

```
  /// @dev the waiting period before accepting the pending merkle root; 1 day by
default
  uint256 public waitingPeriod;
```

```
  function initialize(
    address _admin,
    address _manager,
    address _bot,
    address _pauser,
    address _funder,
    address _token
  ) external initializer {
    ...
    lastSetTime = type(uint256).max;
@>  waitingPeriod = 6 hours;
    ...
  }

  function changeWaitingPeriod(uint256 _waitingPeriod) external onlyRole(MANAGER)
whenNotPaused {
@>  require(_waitingPeriod >= 6 hours && _waitingPeriod != waitingPeriod, "Invalid
waiting period");
    waitingPeriod = _waitingPeriod;
    emit WaitingPeriodUpdated(_waitingPeriod);
  }
```

Recommendation

Consider updating either the initializer default or the NatSpec comment so that the documented default waiting period matches the value enforced on-chain.

Alleviation

[CertiK, 08/24/2026]:

The team heeded the advice and resolved the issue in commit b67efb6.

LDS-15 | One Consumed Row Can Revert Unrelated Batch Operations

Category	Severity	Location	Status
Logical Issue	Informational	InterestDistributor.sol (07/27-50c8c4): 129~135; LockedEarnPool.sol (07/27-50c8c4): 190~198	Resolved

Description

`InterestDistributor.batchClaim` and `LockedEarnPool.batchRequestMaturityWithdraw` both process multiple independent user rows by looping into strict single-row handlers. If any one row has already been consumed before the batch executes, the single-row handler reverts and the entire batch transaction is reverted, delaying unrelated users included later in the same batch.

In `InterestDistributor`, `batchClaim` loops into `claim`:

```
for (uint256 i = 0; i < _accounts.length; i++) {
  claim(_accounts[i], _totalAmounts[i], _proofs[i]);
}
```

`claim` reverts when the cumulative amount has already been claimed:

```
uint256 claimedAmount = claimed[_account];
require(_totalAmount > claimedAmount, "Invalid total amount");
```

Because both `claim` and `batchClaim` are permissionless, an external actor can copy a valid row from a pending public batch transaction and submit it through `claim` first. The targeted account receives its correct interest, but the original batch then reverts when it reaches that already-claimed row.

A similar operational issue exists in `LockedEarnPool.batchRequestMaturityWithdraw`. The BOT-only batch function loops into `_requestMaturityWithdraw`:

```
for (uint256 i = 0; i < users.length; i++) {
  _requestMaturityWithdraw(users[i], posIds[i]);
}
```

The internal handler reverts if the position has already been closed:

```
Position storage pos = userPositions[user][posId];
require(pos.principal > 0 && !pos.closed, "invalid position");
```

A user included in the BOT's pending maturity-withdraw batch can front-run with their own `requestMaturityWithdraw(posId)`. This closes their position and queues their withdrawal correctly, but causes the BOT

batch to revert when it later reaches the same `(user, posId)` row.

Impact

This does not allow theft or reward redirection. The consumed row is paid or queued for the correct beneficiary.

The practical impact is operational denial of service against batch automation:

- gas wasted by the batch submitter;
- unrelated users in the same batch are delayed;
- the operator or BOT must rebuild and resubmit the batch without the already-consumed row.

The `InterestDistributor.batchClaim` case is broader because any third party can copy and consume a public Merkle claim row. The `LockedEarnPool.batchRequestMaturityWithdraw` case is narrower because only the position owner can preempt their own row, and the batch function itself is restricted to `BOT`.

Recommendation

Make batch paths idempotent only for rows that are valid but already consumed.

For Merkle claims, verify the proof first, then skip only equal cumulative totals. Revert on lower totals or invalid proofs.

For maturity withdrawals, skip only positions already consumed by a prior maturity-withdraw request, and keep reverting for invalid BOT list entries.

Skipped rows must never transfer funds or enqueue principal again.

Alleviation

[CertiK, 08/24/2026]:

The team heeded the advice and resolved the issue in commits 5d27476 and 6b3eb22, c720cbd.

LDS-16 Incompatibility With Deflationary Tokens (Non-Standard ERC20 Token)

Category	Severity	Location	Status
Volatile Code	Informational	CreditFundBase.sol (07/27-50c8c4): 160~168; FlexEarnPool.sol (07/27-50c8c4): 59~60; InterestDistributor.sol (07/27-50c8c4): 115~120; LockedEarnPool.sol (07/27-50c8c4): 118~129, 294~307; SurfinAdapter.sol (07/27-50c8c4): 132~134, 162~166	Acknowledged

Description

The Surfin contracts assume that ERC20 transfers move exactly the requested `amount`. Several flows update accounting using the nominal input amount without checking the recipient's actual balance delta. If the configured asset token is fee-on-transfer, deflationary, rebasing, or otherwise non-standard, the contracts can over-credit internal accounting relative to the real tokens received, which can cause later transfers, claims, or settlement operations to fail or underpay users.

For example, `CreditFundBase.finishWithdraw` pulls `amount` from the adapter and then increases `withdrawQuota` by the same `amount`:

```
IERC20(asset).safeTransferFrom(msg.sender, address(this), amount);
withdrawQuota += amount;
```

If the pool receives less than `amount`, `withdrawQuota` will be overstated. The pool may then confirm withdrawal batches that are not fully backed by its real token balance, causing later `claimWithdraw` calls to revert or underpay users depending on token behavior.

The issue is constrained if the system is intended to support only standard USDT-like tokens with exact transfer semantics. However, the contracts do not enforce this assumption on-chain, so deploying with or upgrading to a non-standard asset would break accounting invariants.

Recommendation

Document and enforce that Surfin only supports standard ERC20 tokens with exact transfer semantics. Reject fee-on-transfer, rebasing, or deflationary tokens, or update accounting based on actual balance deltas instead of nominal input amounts.

Alleviation

[Lista Dao, 08/14/2026]:

Issue acknowledged. I won't make any changes for the current version.

LDS-17 Front-Running Withdrawal Cancellation Can Revert BOT Funding

Category	Severity	Location	Status
Logical Issue	Informational	CreditFundBase.sol (07/27-50c8c4): 167	Partially Resolved

Description

`CreditFundBase.finishWithdraw()` is used by the adapter to fund pending withdrawal batches. When `amount > 0`, it transfers funds from the adapter, increases `withdrawQuota`, and checks that the pool has not been overfunded:

```
withdrawQuota += amount;
require(withdrawQuota <= totalPendingWithdraw, "quota exceeds pending");
```

In `FlexEarnPool`, users can cancel unconfirmed withdrawal requests. The cancellation path removes the request and reduces both the batch amount and `totalPendingWithdraw`, but it does not adjust `withdrawQuota`.

As a result, a BOT funding transaction can become stale between submission and execution. If the BOT submits `finishWithdraw(100)` while `totalPendingWithdraw` is `100`, a user can front-run it by cancelling a `40` withdrawal request. The pending amount becomes `60`, but the BOT still attempts to fund `100`, causing the quota check to fail.

Impact

An unprivileged Flex user can intentionally make a BOT funding transaction revert by cancelling a pending withdrawal before it executes. This wastes BOT gas and can temporarily delay confirmation for other users in the same withdrawal batch.

The impact is limited: no funds are lost, withdrawals are not permanently stuck, and the BOT can recover by recalculating the required amount and resubmitting.

Recommendation

Make `finishWithdraw()` use the current pending amount at execution time. If the BOT passes a stale amount, only accept up to the remaining fundable amount:

`totalPendingWithdraw - withdrawQuota`

Avoid simply reducing `withdrawQuota` on cancellation unless the excess funds are also refunded or separately accounted for.

Alleviation

[Lista Dao, 08/23/2026]:

The team heeded the advice and partially resolved the issue in commit ee9fc63576d5d50c42e54ccfb6f2c47c990767b4.

LDS-18 | BOT Renewal Bypasses Entry-Side Controls

Category	Severity	Location	Status
Design Issue	Informational	LockedEarnPool.sol (07/27-50c8c4): 246	Partially Resolved

Description

`renewPosition` is a BOT-only path that rolls a matured auto-renew position into a new locked position. Unlike the other entry-like paths (`deposit` and `reinvest`), it does not apply `whenNotPaused` , `whenDepositNotPaused` , the target cohort `depositDeadline` , or the current `minDeposit` check.

A reachable sequence is: a user deposits with `autoRenew = true` , the original cohort matures, the manager or pauser enables `depositPaused` or pauses the pool, or the target cohort's deposit window has already closed, and the BOT calls `renewPosition` into an enabled cohort. The function closes the matured position and appends a new locked position even though protocol controls may be intended to stop new locked exposure, halt subscriptions into closed cohorts, or route matured positions to an exit flow during emergency or wind-down operations.

The same path also bypasses the current minimum deposit invariant. If `minDeposit` is raised after a user already holds an auto-renew position with smaller principal, the BOT can still roll that principal into a new cohort and create a sub-minimum locked position. This can leave operational dust or violate the pool's current minimum-size policy.

The impact is constrained because only the globally assigned BOT can trigger the transition, the user previously opted into auto-renewal, no additional principal is created, no funds move, and the renewed position has `autoRenew` forced off. The issue is that pause, wind-down, subscription-window, and minimum-size controls cannot prevent this BOT-driven renewal path when operators intend to halt or constrain new locked exposure.

Recommendation

We would like to double confirm whether current design is as intended.

Alleviation

[CertiK, 08/24/2026]:

`renewPosition()` now has the same pause, wind-down, and deposit-window checks as `deposit()` / `reinvest()` , so BOT cannot roll into a new cohort while the pool is paused, deposits are paused, or the target window is closed.

It still does not check `minDeposit` , so a position smaller than the current minimum can still be renewed.

The team partially resolved the issue in commits 6d12c34 and 602a9a6.

LDS-19 Early Redemption Payout Is Fixed Even If Funding Arrives After Maturity

Category	Severity	Location	Status
Design Issue	Informational	CreditFundBase.sol (07/27-50c8c4): 278, 309; LockedEarnPool.sol (07/27-50c8c4): 154, 163	Acknowledged

Description

Locked early-redemption requests permanently store the payout calculated at request time. When `requestEarlyRedeem()` is called inside the penalty window, it passes the already-penalized payout into `_enqueueWithdraw()`, and the stored `WithdrawalRequest.amount` is later returned unchanged by `_consumeConfirmedWithdraw()`.

The protocol tests document a different maturity invariant: if an early-redemption request remains unfunded until the position matures, the penalty should be voided and the withdrawal should convert to a normal maturity withdrawal. The current queue stores only the payout amount and never revalues the request at confirmation or claim time.

`test/surfin/SurfinConflict.t.sol`:

```
/* ============================================================================
   * CONFLICT-2 — early-redeem penalty is locked in and never reversed at maturity.
   *
   * PRD §4.4: if an early-redeem request is left unfunded until the position's own
   *           maturity (T-30 checkpoint), the penalty is VOIDED — the request
converts
   *           to a normal maturity withdrawal (full principal) and base interest is
   *           back-filled for the whole term.
   * CONTRACT: requestEarlyRedeem computes the penalized payout once, at request
time,
   *           and enqueues it. There is no maturity-reversal path, so the penalty
   *           stands no matter how long funding takes.
   * ============================================================================*/

  function test_conflict2_earlyRedeemPenaltyStandsPastMaturity_divergesFromPRD()
public {
```

This is practically reachable when adapter funding is delayed past the source cohort maturity. Affected users receive the stale penalty-reduced principal even though the maturity condition that should remove the penalty has already occurred.

Recommendation

We would like to double confirm whether the current design is as intended.

Alleviation

[Lista Dao, 08/18/2026]:

Issue acknowledged. I won't make any changes for the current version.

LDS-20 | Early Redemption Does Not Forfeit Merkle-Distributed Interest

Category	Severity	Location	Status
Logical Issue, Design Issue	Informational	LockedEarnPool.sol (07/27-50c8c4): 16~18	Acknowledged

Description

`requestEarlyRedeem` reduces only the principal payout through `_earlyRedeemPayout` and does not record forfeited interest eligibility in a state that `InterestDistributor` can enforce. `InterestDistributor.claim` validates only the Merkle proof and the user's previously `claimed` amount; it does not check the user's position or early-redemption state.

After an interest Merkle root containing a user's cumulative entitlement is accepted, the user can request early redemption and then claim the entitlement using the already-valid proof. A root accepted before redemption cannot be corrected to prevent this claim.

This allows early redeemers to receive interest despite the documented invariant that all interest is forfeited on early redemption.

```
/**
 ...
 * shares the same schedule and one update covers them all. Early redemption
 * (partial allowed) forfeits all interest and, if made within PENALTY_WINDOW of
 * deposit, deducts a flat `penaltyRate` on the redeemed principal
 ...
 */
contract LockedEarnPool is CreditFundBase {
```

```
  function _earlyRedeemPayout(uint256 depositTime, uint256 amount) internal view
returns (uint256) {
    if (block.timestamp < depositTime + PENALTY_WINDOW) {
      uint256 penalty = (amount * penaltyRate) / PRECISION;
      return amount - penalty;
    }
    return amount;
  }
```

Scenario

A user whose cumulative interest entitlement has been included in an accepted Merkle root requests early redemption, receiving the principal payout reduced only by `_earlyRedeemPayout`. The user can then submit the previously valid Merkle proof to `InterestDistributor.claim` and receive the included interest because the claim logic does not check whether the user has redeemed early.

Recommendation

We would like to double confirm with the client whether this situation would occur in actual practice.

Alleviation

[Lista Dao, 08/18/2026]:

Issue acknowledged. I won't make any changes for the current version.

LDS-21 Cancelling Partially Funded Flex Withdrawals Can Temporarily Park Adapter Liquidity In The Flex Pool

Category	Severity	Location	Status
Logical Issue, Design Issue	Informational	FlexEarnPool.sol (07/27-50c8c4): 90~95	Resolved

Description

Flex withdrawals use a batch-based funding flow. When a user calls `requestWithdraw`, their Flex LP is burned and the requested amount is added to the current withdrawal batch and `totalPendingWithdraw`.

The adapter can later call `finishWithdraw` with a partial amount. In that case, assets are transferred from the adapter into the Flex pool and `withdrawQuota` is increased, but the batch is not confirmed until `withdrawQuota` is sufficient to cover the full batch amount.

If the user cancels the still-unconfirmed withdrawal after partial funding, `_removeWithdrawRequest` only removes the request from the batch and decreases `totalPendingWithdraw`. `cancelWithdraw` then remints the user's LP. However, the cancellation does not decrease `withdrawQuota` and does not return the already transferred assets from the Flex pool back to the adapter.

As a result, adapter liquidity can become temporarily parked in the Flex pool without a matching pending Flex withdrawal obligation.

For example:

1. A user requests to withdraw `100 USDT`.
2. The BOT partially funds the Flex batch with `70 USDT`.
3. The batch remains unconfirmed because `70 < 100`.
4. The user cancels the withdrawal.
5. The user receives back `100` Flex LP.
6. The `70 USDT` remains in the Flex pool and `withdrawQuota` remains increased.

This does not make the system insolvent and does not allow the user to steal funds. The parked liquidity can later be consumed by future Flex withdrawals. However, while it remains in the Flex pool, it is no longer counted in `SurfinAdapter.idleBalance()` and therefore cannot be used by shared adapter liquidity flows such as locked withdrawal funding, interest funding, or deployment to Surfin.

This is reachable during normal operation when the BOT performs partial Flex withdrawal funding and the requester cancels before the batch is confirmed.

Impact

A user can cause adapter liquidity to be temporarily unavailable for non-Flex uses after a normal partial funding operation. This can reduce liquidity available for locked withdrawals, interest funding, or deployment, creating a practical liquidity fragmentation / availability issue.

Root Cause

Cancellation of an unconfirmed Flex withdrawal removes the pending obligation and restores the user's LP, but does not reconcile any already-funded global `withdrawQuota` or return surplus assets to the adapter.

Relevant code:

```
function cancelWithdraw(uint256 idx) external whenNotPaused nonReentrant {
    uint256 amount = _removeWithdrawRequest(msg.sender, idx);
    _mint(msg.sender, amount);

    emit CancelWithdrawal(msg.sender, idx, amount);
}
```

```
function _removeWithdrawRequest(address user, uint256 idx) internal returns (uint256
amount) {
    ...
    totalWithdrawAmountInBatch[req.batchId] -= req.amount;
    totalPendingWithdraw -= req.amount;
    amount = req.amount;
}
```

```
function finishWithdraw(uint256 amount) external {
    require(msg.sender == adapter, "only adapter can call");

    if (amount > 0) {
        IERC20(asset).safeTransferFrom(msg.sender, address(this), amount);
        withdrawQuota += amount;
        require(withdrawQuota <= totalPendingWithdraw, "quota exceeds pending");
    }

    ...
}
```

Recommendation

We would like to double confirm whether current design is as intended.

Alleviation

[CertiK, 08/24/2026]:

The team resolved it by returning surplus cash to the adapter on cancel. `_removeWithdrawRequest()` now compares `withdrawQuota` to the unconfirmed obligation; any excess is transferred back so partial funding cannot stay parked in the pool after the pending withdrawal is gone.

Changes have been reflected in the commit d4a1690.

LDS-23 | Flex Withdrawals Can Revert Stale Interest Funding

Category	Severity	Location	Status
Denial of Service	Informational	src/surfin/SurfinAdapter.sol (08/18-ee9fc6): 211	Partially Resolved

Description

`SurfinAdapter.fundInterest(uint256)` checks a manager-selected funding amount against the live `_availableForInterest()` value. This value is `freeIdle()` minus the Flex pool's `onDemandUnfunded()` obligation.

A Flex LP holder can change that ceiling after the manager prepares the funding transaction. `FlexEarnPool.requestWithdraw(A)` burns LP tokens and increases `totalPendingWithdraw` by `A`, but it does not move assets or increase funded withdrawals. This can increase `onDemandUnfunded()` and make a previously valid `fundInterest(X)` call revert with `insufficient idle`.

The user can leave the withdrawal request unconfirmed until the manager transaction fails, then cancel it and allow the same funding amount to succeed on retry. Source-default zero `minWithdraw` and `dailyLimit` values allow repeated request and cancel attempts, while a configured `dailyLimit` bounds repetition.

This does not show asset extraction, a permanent lock, or a durable outage. The impact depends on deployed withdrawal limits, how often interest funding is time sensitive, and whether managers commonly fund amounts close to the live available ceiling.

Scenario

A manager submits `fundInterest(1000)` when `freeIdle()` is 1000 and there are no unfunded flex withdrawals. Before it executes, a flex LP holder calls `requestWithdraw(1)`, which burns 1 LP token and adds 1 to pending withdrawals without reducing idle assets or increasing funded withdrawals. `onDemandUnfunded()` becomes 1, reducing `_availableForInterest()` to 999, so the manager's transaction reverts with `insufficient idle`; the holder can subsequently cancel the unconfirmed withdrawal to restore the prior funding capacity.

Proof of Concept

Attack path 1 — one-unit sharp case:

1. Set `freeIdle() = 1000`, with no unfunded flex withdrawal queue (`pending == funded`).
2. The manager prepares `fundInterest(1000)`.
3. Before it executes, a flex LP holder calls `requestWithdraw(1)`. No assets leave the adapter, but pending withdrawals increase by 1 and the live interest ceiling becomes 999.
4. `fundInterest(1000)` reverts with `insufficient idle`.
5. The holder may cancel the still-unconfirmed request, restoring the pending amount and LP position; a retry of `fundInterest(1000)` then succeeds.

Attack path 2 — general blocking amount:

1. At transaction preparation, record `B = freeIdle()`, `P = pending`, `F = funded`, and a funding amount `X <= B-max(P-F,0)`.
2. A pre-existing LP holder requests `A = B-X+F-P+1` units, provided that amount is within the holder's balance and withdrawal limits.
3. The request changes the live unfunded obligation to `max(P+A-F,0)` without changing `B`, causing `fundInterest(X)` to revert whenever `X > max(B-max(P+A-F,0),0)`.
4. Keep the request unconfirmed through the funding attempt, then cancel it to return excess quota and repeat on subsequent pre-sized top-ups.

Recommendation

Make interest funding resilient to permissionless changes in unconfirmed Flex withdrawal obligations. For example, size funding from current state at execution, use conservative buffers, or introduce a preparation/finalization design that prevents unconfirmed withdrawal changes from invalidating a committed funding action.

Alleviation

[Lista Dao, 08/25/2026]: Acknowledged, the cancelWithdraw logic has been removed in commit:

https://github.com/lista-dao/lista-new-contracts/tree/b306c579c00b9c2a24f39aad61b9ca1807d19a8d.

[CertiK, 08/25/2026]: The team has partially resolved this finding. The cancellation-based repeatability described in the original finding no longer applies because `cancelWithdraw` has been removed. However, the stale-transaction griefing vector remains: a Flex LP holder can still submit `requestWithdraw` before a pending `fundInterest` transaction executes, increasing `onDemandUnfunded` and reducing `_availableForInterest`, which may cause a previously valid `fundInterest` amount to revert with "insufficient idle".

LDS-24 | Renewed Positions Can Bypass Auto-Renew Cap

Category	Severity	Location	Status
Logical Issue	Informational	src/surfin/LockedEarnPool.sol (08/18-ee9fc6): 296, 303, 336, 377	Acknowledged

Description

Auto-renew is documented as limited to one rollover, but users can enable it again after a renewal or reinvestment.

```
/**
* @dev roll a matured auto-renew position's principal into a fresh cohort.
*      Only principal is rolled (interest is distributed separately). Renewal
*      is limited to one term: the new position has auto-renew forced off.
*      Driven by the settlement-day job (BOT).
* @param user the position owner
* @param posId the matured position id
* @param newCohortId the cohort to renew into
*/
```

Renewed and reinvested positions are created with `autoRenew: false`, but the contract does not store durable eligibility state such as a renewal depth or a `canAutoRenew` flag. As a result, `toggleAutoRenew` can set `autoRenew` to true for any open position outside the T-32 lock window, including positions that were created by `renewPosition` or `reinvest` with auto-renew forced off.

After the BOT renews a user's position into a new cohort, the new position will usually have more than 32 days before maturity. The user can call `toggleAutoRenew(newPosId)` to enable auto-renew again, allowing the BOT's normal maturity processing to renew it another time. This bypasses the stated one-term cap and can keep principal rolling through repeated terms contrary to the product and settlement assumptions.

Recommendation

We would like to double confirm whether current design is as intended.

Alleviation

[Lista Dao, 09/15/2026]: The behaviour is intended. `autoRenew: false` on the new position is there so every further rollover is a deliberate opt-in by the holder, not a cap on how many times they may opt in. For `reinvest` the payout was already claimable to the wallet, so rolling it forward is a fresh deposit rather than a continuation.

Only the stale comments were corrected, in `0db2041`.

LDS-25 | Auto-Renew Positions Lack Maturity Withdrawal Fallback

Category	Severity	Location	Status
Design Issue	Informational	LockedEarnPool.sol (07/27-50c8c4): 246	Acknowledged

Description

Users who leave auto-renew enabled lose their direct maturity-withdrawal fallback after the T-32 lock window. If renewal is unavailable, their principal remains in an open matured position and cannot be queued through the user or BOT maturity-withdraw paths.

A user can reach this state by depositing with `autoRenew = true` or by enabling it before the lock window. At or after maturity, `_requestMaturityWithdraw` reverts because `pos.autoRenew` is still true. `batchRequestMaturityWithdraw` skips the position with `SkipReason.AutoRenew`, and `toggleAutoRenew` cannot clear the flag because `block.timestamp + AUTO_RENEW_LOCK_WINDOW` is no longer below the cohort maturity. Extending the original cohort after maturity also does not restore the toggle path because `setCohort` limits drift to `MAX_ALIGN_WINDOW`, which is shorter than the 32-day lock requirement.

The only normal recovery path is BOT-only `renewPosition`, which depends on renewal being available for the position. A later BOT can still recover the position by renewing it into a valid cohort when deposits are open, so this is not permanent loss in every case. The issue becomes a principal-exit liveness failure during wind-down or `depositPaused`, when no valid renewal cohort is provided, or when BOT execution remains unavailable.

Recommendation

We would like to double confirm whether current design is as intended.

Alleviation

[Lista Dao, 08/25/2026]: Intended, with the exposure narrowed operationally. The routine triggers are covered off-chain: the next cohort is created every month at T-3 for every chain and term, and the settlement job rolls forward when a target deposit window has closed, retries any row it skipped, and alerts on BOT permission or key failure.

LDS-26 Discussion On Upgrading To New Implementation Can Zero New Storage Variables

Category	Severity	Location	Status
Design Issue, Coding Issue	Informational	src/surfin/CreditFundBase.sol (08/18-ee9fc6): 95	Acknowledged

Description

The new commit ee9fc63576d5d50c42e54ccfb6f2c47c990767b4 adds two storage variables by shrinking each contract's `__gap` . On a proxy still running the old implementation those slots were never written, so after the scripted bare `upgradeToAndCall(newImpl, "")` they read zero while the surrounding legacy state keeps its old values:

- **Pools (`totalConfirmedUnclaimed = 0`)**: any funded-but-unclaimed withdrawal request (batch already confirmed, cash already in the pool) then hits `totalConfirmedUnclaimed -= amount` as `0 - amount`, then hitting arithmetic panic. Every `claimWithdraw`/`reinvest` of a legacy confirmed payout reverts, with no in-protocol repair (the counter has no setter and cannot be re-initialized). Later fundings raise the counter again, so an older claim can silently consume accounting owed to newer claimants who then panic. The adapter's `_poolUnfunded` also overstates the unfunded queue, inflating the hard floor and shrinking the deploy/interest caps.

```
  // principal belonging to batches already confirmed but not yet claimed. Splits
  // `totalPendingWithdraw` into its two halves:
  //     totalPendingWithdraw = unconfirmed obligation + totalConfirmedUnclaimed
  // Only the unconfirmed half still needs cash from the adapter — the confirmed
half is
  // already sitting in this contract and must not license further quota. Declared
last
  // (rather than next to totalPendingWithdraw) so no existing storage slot shifts;
one
  // slot is taken from __gap below.
  uint256 public totalConfirmedUnclaimed;

  ...

  function _consumeConfirmedWithdraw(
    address user,
    uint256 idx,
    uint256 expectedAmount
  ) internal returns (uint256 amount) {
    ...
    totalConfirmedUnclaimed -= req.amount;
    amount = req.amount;
  }
```

- **Distributor (`pendingActivationTime = 0`)**: the accept gate `block.timestamp >= pendingActivationTime` becomes vacuously true, so a BOT can accept a staged root in the same block as the upgrade — voiding the ≥6-hour review window this changeset specifically added.

```
  /**
   * @dev earliest timestamp `pendingMerkleRoot` may be accepted at, frozen when the
root
   *      is staged. 0 when nothing is pending.
   *
   *      Computing acceptance live as `lastSetTime + waitingPeriod` would let a
   *      `changeWaitingPeriod` landing mid-flight retroactively move the deadline
of a
   *      root already under review — shortening the window the delay exists to
provide,
   *      or pushing an urgent root days out by accident. Each root instead carries
the
   *      period it was staged under, so `changeWaitingPeriod` only affects roots
staged
   *      after it.
   *
   *      Declared last so no existing storage slot shifts; one slot is taken from
__gap.
   */
  uint256 public pendingActivationTime;

   ...

   function acceptMerkleRoot() external onlyRole(BOT) whenNotPaused {
    ...
    require(block.timestamp >= pendingActivationTime, "Not ready to accept");
    ...
  }
```

Recommendation

We would like to confirm if the proxies are already running the old implementation. If so, we highly recommend adding additional migration logic to prevent similar inconsistent state variable issues.

Alleviation

[Lista Dao, 08/25/2026]: The surfin contracts have not been deployed yet.

[CertiK, 08/26/2026]: Thank you for the clarification. Since the Surfin contracts have not been deployed yet, this issue does not currently impact any live proxy state.

As a reminder for future upgrades, when adding new storage variables to an upgradeable contract, the storage gap should be reduced only by the exact number of newly consumed storage slots. For example, adding one new uint256 variable to a contract with a uint256[50] __gap should normally reduce the gap to uint256[49]. Reducing it further may shift child contract storage variables in a future upgrade scenario and cause storage layout incompatibility.

We recommend generating and comparing storage layout reports before any future proxy upgrade.

LDS-28 `settleRecall` Uses A Stale Floor And Rejects Fully Funded Recalls

Category	Severity	Location	Status
Logical Issue	Informational	CreditFundBase.sol (07/27-50c8c4): 170, 215; SurfinAdapter.sol (07/27-50c8c4): 165, 167, 389, 420	Pending

Description

A locked withdrawal settlement can revert even when the manager supplies enough USDT to fund the locked queue and fees. This can delay principal claims despite sufficient post-settlement reserves.

`settleRecall` receives `recalledAmount`, adds `feeAmount` to `accruedFee`, and calls `_finishLockedWithdraw`. Its availability check subtracts the current `hardFloor()`, whose base still includes the queue being funded. Funding would immediately reduce this unfunded obligation through `withdrawQuota`, even before batch confirmation; confirmation then moves the funded amount into `totalConfirmedUnclaimed`. However, the availability check runs before either update.

For example, assume a 100,000 USDT locked position has matured, no other principal remains, and the existing floor has been consumed through permitted interest funding. Recalling 100,000 USDT and attempting to fund the entire queue still reserves the pre-settlement 3% floor, making only 97,000 USDT available. The call therefore reverts even though completing settlement would reduce the floor to zero. Operators can instead split funding across repeated calls as the floor decreases, or provide additional liquidity or adjust the floor rate. This adds operational overhead and can delay claims.

Recommendation

Validate locked recall settlements against the reserve required by the resulting post-settlement state, so the liability being funded is not also counted as an unfunded floor obligation during the same settlement.

APPENDIX | LISTA DAO (SURFIN) - AUDIT

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Denial of Service	Denial of Service findings indicate that an attacker may prevent the program from operating correctly or responding to legitimate requests.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.